EPFL — Fall 2026

MATH-310

Computer Security and Privacy

Prof. Thomas Bourgeat

---

by Arthur Herbette

Lecture notes — Monday 7 September 2026

This document was originally created for my own use, but I am sharing it because I believe it may be useful to others for studying and reference purposes. However mistake can happen and there will inevitably be some errors, if you found any, you are more than welcome to send me a message (Arthur Hbe on telegram) or to directly do a PR on the repo

`https://github.com/Arthur926564/LectureNotes`

If you think that something is missing or that something is not clear, I will be more than happy to add your modification to this document.

Unless otherwise stated, all material in these notes is based on the course content, including the diagrams, explanations, and examples.

Note also that these notes were taken during the lecture, but not only; some materials was added later (that's why you may read remarks that sounds like I knew the course before taking it).

Finally, thanks a lot to Joachim Favre for his help with the setup and whose style is used in this document.

Contents

1 Introduction **5**
1.1 Basic concepts of security engineering 6

2 Applied Cryptography **9**
2.1 Introduction 9
2.1.1 The Core Problem For Today 9
2.1.2 The origins of cryptography: the quest for confidentiality 9
2.2 Adversaries in Cryptography 11
2.3 Perfect Secrecy and One-Time-Pad 12
2.4 From OTP to Stream Ciphers 14
2.4.1 Interlude on building KSG 15
2.5 Public Key Cryptography 16
2.5.1 Authenticity 18
2.5.2 Hash Functions 19
2.5.3 Integrity 20
Integrity for Symmetric Cryptography 20
2.6 Authentication 24
2.6.1 Secure checking 27
2.6.2 Tokens 29
2.7 Adversarial thinking, Attacks and defenses 30
2.7.1 Reasoning as a defender 32

Introduction

Overview of the course

The goal of the course is to

- Understand basic concepts and principles of security design and engineering that will **outlast current technology**.
- Learn to **model threats** and **think critically** about security problems

  adversarial thinking

- Get familiar with a number of security mechanisms, learn their purpose and limitations building a **toolbox for security engineering**

Chapter 1

Introduction

What is computer security

The first question that we'll answer is what are we studying in this course, what is this course about

> **Definition 1.** (Computer Security) **Properties** of a computer system must hold in the presence of a resourced strategic adversary.

What Properties?

Traditional properties are such

- **Confidentiality** — prevention of unauthorized disclosure of information (e.g. The adversary should not be able to read my bank statement)
- **Integrity** — prevention of unauthorized modification of information (e.g. The adversary should not be able to change my bank balance)
- **Availability** — prevention of unauthorized denial of service (e.g. The adversary should not prevent me accessing my bank account)

But there is also not-so-traditional (But important properties)

- Authenticity
- Anonymity
- Isolation
- Non-repudiation
- ⋮

The Security policy

> **Definition 2.** Security policy A high-level description of the **security properties** that must hold in the system in relation to **assets** and **principals**.

- **Assets** (objects): anything with value that needs to be protected
- **Principals** (subjects): people, computer program, services

The resources strategic adversary

Coming back on the definition of computer security, what do we mean by '*resourced strategic adversary*'? To do so we need to define the **threat model**

> **Definition 3.** Threat model Describes the **resources** available to the **adversary** and the adversary's capabilities (observe, influence, corrupt, ...)

- The **adversary** is a malicious entity aiming at breaching the security policy
- The **adversary** is **strategic**: the adversary will choose the **optimal** way to use her resource to mount an attack that violates the security properties.

(Threat modelling is a very hard task!)

Here we have defined a lot of key vocabulary to talk about the adversary but we miss a few and here are those:

Definition 4. Threat What is the feared event, the goal of the adversary that we don't want materialized

For instance

A hacker wants to retrieve money breaking into the bank's system
A student wants to learn my password by looking over my shoulder

Definition 5. Vulnerability Specific weakness that could be exploited by adversaries with interest in a lot of different assets

For instance

The banking API is not protected. The password appears in plain text in my screen.

Definition 6. Harm The bad thing that happens when the threat materializes

For instance

The adversary steals money, he blocks access to the bank...

All together this give if we were to expand the computer security definition

Properties define by a security policy of a computer system must hold in presence of a resource strategic adversary described by the threat model

If the **properties** hold within the **threat model**, it means that there are no **Vulnerabilities** that can be exploited to materialize threats and no harms happens.

1.1 Basic concepts of security engineering

Securing a system

In order to secure a system one needs a **security mechanism**

Definition 7. Security mechanism Technical mechanism used to ensure that the security policy is not violated by an adversary within the threat model

This can be build with software (programs) + Hardware + Maths (cryptography) & distributed systems, people and procedures. Security mechanisms can be engineered!

Example 1

Policy:ensure the log of transactions is not tampered with by a single employee
Mechanism: encrypt the message before sending

Systems are big! We need security mechanism**S**

Security does not necessarily increase linearly with the number of mechanisms. We could create hundred of walls with locked doors while hiding the key below the entrance carpet.

How do we show systems are secure?

Here we can see that there is a big asymmetry between the attacker and the defender. On the attacker case, we only need **one way** to violate **one** security property (within the threat model). The defender on the other hand need that **no adversary strategy** can violate the security policy.

Therefore one can see that the question '*is this system secure?*' is not enough, the correct question is

Is this system secure under **this** threat model?

A system is said 'secure' if an adversary constrained by a **Specific threat model** cannot violate the **security policy**.

Then in order to show that systems are secure we need arguments:

Definition 8. Security argument rigorous argument that the security mechanisms in place are indeed effective in maintaining the security policy (verbal of mathematical).

Subject to the assumptions of the threat model

For a threat model to be useful, the model must constrain the adversary, otherwise we cannot make a security argument. One could just state: *the adversary know everything and can do anything.*

Security engineering

In security engineering we have three steps

1. High-level specification
   - Define the architecture of the system (e.g., high level block diagram)
   - Define the security policy (principals, assets, security properties)
   - Define the threat model
2. Security design
   - Select/Design security mechanisms
   - State your security argument: which controls maintain which properties?
3. Secure implementation
   - **Implement** mechanisms
   - Ensure they **conform** to the design model
   - Security testing

Summary

Security problems always involve an adversary.

- The adversary is strategic, will take the most damaging approach
- The adversary's capabilities define a threat model
- Security mechanisms aim at fulfilling a security policy within a threat model
- Showing security implies providing a security argument

Chapter 2

Applied Cryptography

2.1 Introduction

Why cryptography matters?

A lot of data need to move from one place to another while wanting to stay private (i.e. no one can stole the data during the trip). But not only, even data at rest (e.g. your password in your os) need also to not be accessible by a third party. To ensure this, one need to **encrypt** data.

Cryptography allows to free you from physical security, it also reduces the TCB to the confidentiality of the key

What can we do with cryptography

Cryptography is also the building block for:

- **Integrity**: Ensuring data hasn't been tampered with (Hashing/Signatures)
- **Authentication**: Verifying identity (Protocols)
- But also **Anonymity**, etc.

In this first lecture we will focus on the basics of using cryptography for confidentiality

2.1.1 The Core Problem For Today

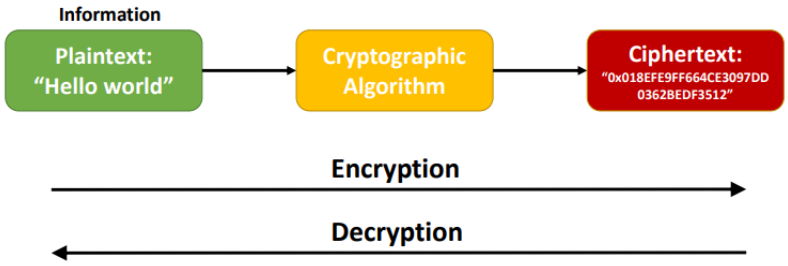
The central problem here is to **secure communication over an insecure channel**. We need a way for Alice to send a message to Bob, so that Eve (the adversary) cannot read it even though Eve have access to the channel where the massing is passing.

Remark 1. confidentiality for data at rest can be seen as a particular case, where Alice = Bob.

2.1.2 The origins of cryptography: the quest for confidentiality

Before going into cryptography, let us first define what are we trying to ensure

Definition 9. confidentiality : information cannot be accessed by unauthorized parties



The **encryption** cannot be reversed without the use of the **key**.

Cryptography as Functions

To have a more formal view of cryptography and being able to ensure confidentiality by proving it mathematically, let us define the encryption/decryption using functions. Here one should define

- **Plaintext** M: Original message.
- **Key**: K Secret value.
- **Ciphertext**: C Scrambled message.

We then have the following equality

$$C = E_K(M)$$
$$M = D_K(C)$$

Cryptographic algorithms for confidentiality

In order to have confidentiality between Alice and Bob, both needs to either encrypt or decrypt the given message that the other is sending, to do so as seen in the equations above, both Alice and Bob need to share the same key K. Having a shared key between Alice and Bob, the algorithm works as follow

- Encrypt message $m \to \text{Enc}(K, M)$
- Send encrypted message $\text{Enc}(K, M) = c$
- Decrypt message $c \to m = \text{Dec}(K, c)$

The core requirement — Invertibility

An statement that we need to make in order to make the encryption/decryption mechanism works is that the decryption function D_K must be the **invert** of E_K :

$$\forall K, M, D_K(E_K(M)) = M$$

If not, Bob cannot recover the original message M.

> **Remark 2.** **Security requirement**: To provide security, these function must also be **hard to invert** without knowing the key K. If inverting a given function could be find meaning for instance that if we knew that the function for encryption was e^x, then it become quite easy to reverse it $\ln(c) = m$.

Hard to invert: The Key Space

What we mean concretely when saying hard to invert is the following: To find the plaintext M, in an ideal cryptosystem, the only viable attack for the adversary (Eve) must be to try every single possible secret key K (a **brute force attack**)

Example 1: Caesar's Cipher

The idea here is simple, we shift every later of the attack by a given number K, the key. The algorithm works as follow

- **Encryption**: Shift each letter by a fixed number K
- **Decryption**: Shift each letter by a fixed number $-K$
- **Key Space**: Only 25 possible keys.

If for instance the plaintext is `ATTACK AT DAWN`, the result Ciphertext is `DWWDFN DW GDZQ`.

Security Insight: $\log_2(25) \approx 4.6$ bits of security. This is **far to small** for real-worls use.

> **Remark 3.** The unit of measure use here $\log_2$ is called the '*bits of security*' which is a logarithmic unit of measurement tat quantifies the **amount of computational effort required to break a cryptographic system**. If a system has n bits of security, it means an attacker must perform roughly 2^n operations (or mathematical steps) to crack it using the best known attack.

Cryptanalysis: The Flaw

- Eve knows that Alice wrote a long-enough message for Bob in English.
- She knows that English most frequent letter is 'e'.
- She can then simply identify the most frequent letter in the ciphertext: it likely corresponds to the letter 'e'.

- She can then directly recover the key (a number between 0 and 25). This is a **frequency analysis attack**.

More than 25 keys: The substitutions Cipher

Here you could think, okay then I will just use more letter like 1000 letters instead of 25 and then my code is secure. To do so, one can use the substitution Cipher. In this case each letter is mapped to a unique, different letter defined by a **permutation** of the 26 letters. What this gives us is a key space of $26! \approx 4.03 \cdot 10^{26}$. $\log_2(26!) \approx 88.4$ bits of security?

However this cipher can still easily be broken using the same frequency analysis as before.

Cryptanalysis: The Flaw, Again

- The same **frequency analysis attack** is effective against the Substitutions Cipher.
- **Vulenrability**: The cipher preserves the statistical properties of the plaintext language (e.g., single-letter frequencies).
- If Eve (the attacker) has a long enough ciphertext C, statistical analysis will break the cipher, rendering the full 26! key space **irrelevant**.
- **Goal of the adversary**: Notice that the goal of the adversary might not be necessarily to recover the full key depending on the scenario, it might only attempt to recover part of the key (the mapping of a subset of letters) .
- Sometimes the goal of the attacker is even smaller: Eve might want only to know wheter Alice sent the message '*Yes*' or '*No*' to Bob.

Bits of security verus Key Space Size

The goal that we would like to reach is that an N-bit key should offer security as close to N bits as possible (i.e. require 2^N attempts). If a 1024-bit key is broken with '*only*' 2^{40} attempts (40 bits of security), the algorithm is considered broken. However as seen in the Caesar, having a bigger key is not always a sign better, we also need to take into account that the adversaries are smart.

2.2 Adversaries in Cryptography

Eve's Power: Passive Eavesdropping

In Caesar's (and the substitution) Cipher, eve's capabilities were very limited, she was only able to overse a long-enough ciphertext C that corresponds to the encryption of a piece of English (and some compute power to test a few keys). Here this is kind of the lowest possible type of adversaries meaning an adversary with lower rights than this can pretty much do nothing. So let us see other possible adversary models

Other Security Models: Type of Adversaries

As seen before, security is a schema that is always measured relative to the power of the **adversary (Eve)**

- **Passive Eavesdropper**: Eve can only read the ciphertext
- **More Active Attacker**: Eve can somehow influence the system by for example convincing Alice to send to Bob very specific messages, observing the precise time it takes Bob tu run its algorithm, etc... anything that can go beyond just observing the communication.

Active Model 1: Known Plaintext Attack (KPA)

The scenario is the following, Eve is given access to multiple pairs (M, C) corresponding to messages and their corresponding ciphertext, all encrypted with the secret key K.

The goal is to go from this pairs to try to guess the key K in order to decrypt other messages.

Why is KPA a Realistic Model?

- **Standard Headers**: Many encrypted messages contain predictable headers (e.g. '*from:*', or as in the film *The Imitation Game*)
- **Malicious Guessing**: If Eve guesses a message (or a part of a message) and observes its encryption, she as an M/C pair.

KPA and the Substitutions Cipher

If we know take the KPA model trying to attack the substitution cipher, it become even easier to crack it. Imagine that Eve knows that $M =$ 'THE QUICK BROWN FOX', she immediately recovers the mapping for multiple letters, drastically reducing the effective key space, without even needing to do statistical analysis study.

Active Model 2: Chosen Plaintext Attack (CPA)

The scenario is the following: Eve can somehow convince Alice (or the system) to encrypt arbitrary messages (chosen by Eve) with the secret key. More abstractly, for any chosen plaintext m, Eve can get access to $E_K(m)$.

Remark 4. We often say that Eve is assumed to have access to an **Encryption Oracle**.

Why is CPA a realistic model?

We can imagine two simple scenarios

- **Alternative Scenario**: Think of a secure messaging service (like Signal or WhatsApp) that uses the sender's key to encrypt the message. If Eve gets limited access to Alice's phone for a few minuts, she can maybe send any piece of data she choose to Bob, to generate an (M, C) pair.
- **Encryption API**: Eve might interact with an encrypted API endpoint where she provides data and receives the encrypted output.

CPA and the Substitution Cipher

Here it is easy to show that the substitution cipher is completely broken by CPA as the attacker only have to ask for the encryption of the alphabet.

Practical Threat Model: Side-Channel Attacks

The scenario is the following: Eve is allowed to not only access an ideal oracle, but also measure physical characteristics of the actual implementation while it is encrypting/decrypting. This allows to have two new attacks:

- **Timing Attack**: Eve measures how long it takes for a given message to get encrypted or a ciphertext to be decrypted.
- **Power Analysis**: Eve observes the energy consumed by the device (e.g., a credit card) doing the crypto.

Why are Side-Channel a Realistic Model?

- **Malicious Ownership**: ou often own a device (like a phone, smart card, or embedded chip) that holds keys belonging to another principal (e.g., Google, your bank) that are meant to be unreadable to you.
- **Concrete Example (DRM)**: The decryption key for Digital Rights Management (DRM) content (like a movie or song) is embedded in your media player or console. By measuring power or timing signals, an attacker could potentially extract that secret key.

2.3 Perfect Secrecy and One-Time-Pad

One-Time-Pad

The key is defined as a string k of **random** bits **as long as the message**. Then one define the encrypted as

$$\text{Enc}(k, m) = m \oplus k$$

Here we have that OTP is **perfectly secure**.

- For any ciphertext C, **every possible M is equally likely**
- The key k is uniformly random, so C gives the adversary **zero new information** about M.

Definition 10. Perfect Secrecy To achieve perfect secrecy for a given encryption algorithm we need the follwing

$$\forall m, c, P(M = m | E_K(m) = c) = P(M = m)$$

Which means that for all message and their ciphertext, the probability of a plaintext M being the message knowing the encryption is the same as the probability of the plaintext M being the message.

Theorem 1. One-Time-Pad achieves perfect secrecy

Proof. *To do so let us work through the LHS, we use that fact that* $m \oplus K = c \implies K = c \oplus m$ *:*

$$\begin{aligned} P(M = m | E_K(m) = c) &= \frac{P(M = m \cap E_K(m) = c)}{P(E_K(m) = c)} \\ &= \frac{P(M = m \cap (m \oplus K) = c)}{P(E_k(m) = c)} \\ &= \frac{P(M = m \cap K = m \oplus c)}{P(K = m \oplus c)} \\ &= \frac{P(M = m) \cdot P(K = m \oplus c)}{P(K = m \oplus c)} = P(M = m) \end{aligned}$$

□

The Key Problem of OTP

OTP requires keys that are **truly random**, **message-length**, and **used only once**. And as securely pre-distributing message-length keys for real-world applications is impractical, OTP is not practical for real-world applications.

Cryptanalysis Question: Key Reuse

Suppose Alice reuses the key K to encrypt **two single-word english message of length 5**, M_1 and M_2, producing C_1 and C_1

$$\begin{aligned} C_1 &= M_1 \oplus K \\ C_2 &= M_2 \oplus K \end{aligned}$$

The goal is to show that even can '*likely*' recover the key and then the two messages M_1 and M_2. To do so let us multiply our two ciphertexts and use the commutativity of the xor operation. Do so so, one can notice that

$$\begin{aligned} C_1 \oplus C_2 &= (M_1 \oplus K) \oplus (M_2 \oplus K) \\ &= M_1 \oplus M_2 \oplus K \oplus K \\ &= M_1 \oplus M_2 \end{aligned}$$

Which reveals where the messages differ, hence one can do a frequency analysis on the ASCII patterns (space (00-) or letter (01-))

OTP: Integrity is NOT included

OTP guarantees **confidentiality** (nobody can read it) however it **does not** guarantee **integrity** (nobody can tamper with it).

Attack Example (Eve flips the first bit of M)

Imagine that Eve flips the first bit of C to get C'.

Bob decrypts $C' : M' = C' \oplus K = (M \oplus K \oplus \Delta) \oplus K = M \oplus \Delta$.

The result is that Eve successfully changed the message (M' is M with one bit flipped) **without knowing *K***

Downsides of One Time Pad

OTP is nice as it gives perfect secrecy, but if we think about it practically, this is not very useful, recall that the key has to be **as long as** the message, so imagine

having to encrypts GiBs of data, one cannot just have a key that long. Also the key has to be pre-shared so one would need to find a way to have a key of the same sized being shared before hand.

Also in general

- Key **must** be random
- Key **cannot** be reused
- No integrity!

2.4 From OTP to Stream Ciphers

Two Main Types of Symmetric Schemes

Definition 11. (Symmetric Schemes) Encryption of plaintext and decryption of ciphertext are done using the **same key**

- **Block Ciphers**: Operate on fixed-size blocks (e.g., 128 bits).
- **Stream Ciphers**: Operate one bit/byte at a time, like a pseudo-OTP. (Example **ChaCha20**)

We also need a fixed-size input to symmetric cryptographic primitives, the size of the key influences the level of security provided

Key properties

- **Known to both parties**: Partners must agree on the key before starting using the primitive
- **It is reused**: The key is pre-shared one (key do have a '*duration*') and then reused
- **It must be secret**: revealing the key eliminates any protection provided by the primitive

Stream Ciphers: The pseudo-OTP

A stream cipher attempt to emulate the OTP's security while solving the key length problem.

- **Short Key** (k): Alice and Bob share a small, manageable key
- **Key stream generator**: Uses K and an **initialization vector (IV)** A to produce an arbitrary long, pseudo-random bit stream S
- **Encryption** $C = M \oplus S$

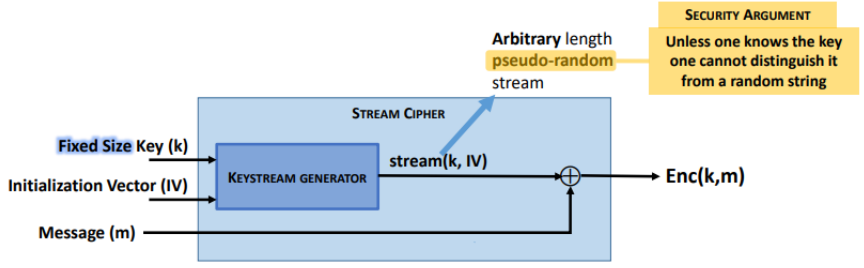
Question: Why does the Key Stream Generator takes in also an IV, and not just the key

Answer

The KSG needs an IV to make sure that the encryption of two different messages will generate two different randomness, without that, one could simply XOR two ciphertexts together to recover the prefix of S, and decode future messages.

KSG: Finite Secret, Infinite '*Randomness*'?

We are generating **potentially infinitely long randomness** (the stream S) from a **finite secret** (K). The security here only hold on the fact that S is being computationally indistinguishable from a truly random OTP key.



(+) Speed	Algorithm are linear in time and constant in space.
(+) Low Error Propagation	Errors in one bit do not affect subsequent symbols.
(-) Low Diffusion	A change in one plaintext bit only affects one ciphertext bit.
(-) Susceptibility to Modification	Low diffusion makes it easier to tamper with the message (like the OTP).

Cryptanalysis: The Problem with Periodicity

Key Stream Generators which have finite state are eventually **periodic**, hence if the message is longer than the period, the stream repeats which allows attacker to exploit it: if the period is identified, and too small, the key stream is known for the entire message.

2.4.1 Interlude on building KSG

Remark 5. You are not expected to fully understand the following subsection, they are here to illustrate some design patterns for KSG, and challenges involved in building secure KSG, and hint at the math needed to study these objects.

The linear Feedback Shift Register Building block for KSG

- **Linear Recurrence Relation**, The next output bit is a linear function (XOR) of previous bits, apply a recurrence relation to extend the initial random sequence of bits, into an arbitrary length sequence of (maybe?) randomly looking bits:
- For example, start with an initial state of 4 bits, $a_0, \ldots, a_3$ and

$$a_n = a_{n-3} \oplus a_{n-4}$$

- Output Sequence (starting from $1, 0, 0, 0$): $1, 0, 0, 0, 1, 0, 0, 1, 1, \ldots$
- **Advantage**: Uses only **shift registers** (for state storage) and **XOR gates** (for feedback), so it is extremely fast and efficient to implement in hardware.

Randomness of LFSR

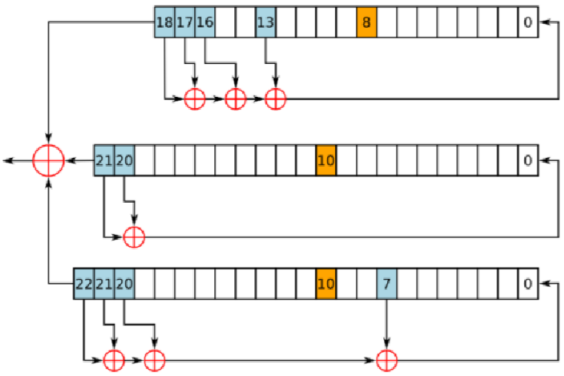
- **Maximal Period**: If the characteristic polynomial (AICC1) of the recurrence relation of the LFSR is primitive (MATH-215, Algebra 3), the maximum possible number of states before repeating: For an L-bit register, the maximum period is $2^L - 1$ states.
- **Distribution Property**: As a consequence, the sequence generated exhibit good distribution properties. Specifically, every possible nonzero state appears exactly once in the cycle, so over one full cycle of values, the output sequence are highly balanced. For example, the sequence will contain exactly 2^{L-1} ones and $2^{L-1} - 1$ zeros.

LFSR — Math Danger

- Despite passing some statistical tests, the underlying operation of an LFSR is purely **linear**
- **Consequence**: Techniques like the Berlekamp-Massey algorithm (beyond the scope of the class) can recover the entire LFSR state and the key stream by observing only a short segment of the output, breaking the cipher.
- **LFSRs should not be used directly as a Key Stream Generators**, though it has been traditionally used as a building block of Key Stream Generators

The A5/1 Cipher

- A5/1: Used historically to secure GSM mobile phone communications.
- It was built by **combining multiple LFSRs** in a non-linear way to try and overcome the linearity flaw while still being cheap and having the nice statistical properties.
- **The Break**: despite the increased complexity, mathematical weaknesses were found, allowing the cipher to be broken much fast than brute force.



Non borken Stream Ciphers

There are alternative, modern stream ciphers for which there are no known practical cryptanalysis. Some example are **Trivium** (based on NLFSRs, variation of LFSRs) and **Salsa20** (using different techniques).

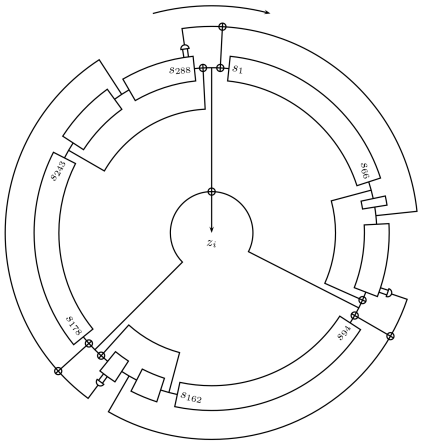


Figure 2.1: Structure of Trivium (Wikipedia)

Bit Security and Consequences

The **cost of breaking**, the goal is to exploit a mathematical property or bug that can break the scheme much faster than $2^{|K|}$ attempts, which therefore **reduce the bit security**. As we can see the difficulty of those scheme is in the design of mathematically secure and subtly resistant primitives which has been found to be incredibly hard.

Best practice Use **well-vetted**, **peer-reviewed**, **standard algorithms and their implementation** (like AES, ChaCha20, SHA-3)

2.5 Public Key Cryptography

The Key Distribution Problem

As seen before, symmetric ciphers are fast, but require a pre-shared secret key. But a more fundamental question come first, if Eve can observe all conversation between Alice and Bob, how can they first agree on a secret key without having Eve to know?

The answer to that is public-key cryptography which allowed two parties who have never met to establish a secure channel. A famous example is Diffie-Hellman

Diffie-Hellman Setup

Alice, Bob (and Eve) know a large public parameters: a prime number p and a generator g

Alice Actions

- Choose a **private** secret a
- Compute the public value $A = g^a \bmod p$
- Sends A to Bob

Bob's Actions

- Chooses a private secret b
- Computes the Public Value $B = g^b \bmod p$
- Sends B to Alice

Remark 6. Eve sees A, B, g and p but cannot find a or b due to the Discrete Log Problem, while it has a low complexity to compute g to the power b modulo p (with fast modular exponentiation algorithm, for example), there are no known fast (polynomial) algorithm to recover a from A and p (or b from B and p).

The Shared Secret

Both Alice and Bob can compute a shared secret K

$$\begin{aligned} K = B^a \bmod p &= \left(g^b\right)^a \bmod p \\ &= g^{ba} \bmod p \\ &= g^{ab} \bmod p \\ &= \left(g^b\right)^a \bmod p = A^b \bmod p \end{aligned}$$

Where but $B^a \bmod p$ and $A^b \bmod p$ are computable by Alice and Bob resp. However Eve cannot compute K ,because she has neither a nor b.

Trapdoor Functions

This example of DF is one of the many examples of how to do public key cryptography to establish a shared secret (a shared key) to then do symmetric cryptography. They boil down to mathematical functions that are **easy to compute** in one direction but **hard** (high complexity) to do in the other direction. Those rely on underlying difficulty, for DH it is the **Discrete Logarithm Problem**.

Beyond DH

There are other ways to set a shared secret (or even to directly communicate), all involve interesting mathematics.

- **RSA**: Based on the factoring problem
- **Elliptic Curve Cryptography (ECC)**: Can do Discrete Log on ECC, instead of on public-key crypto
- **Post-Quantum Cryptography**: new primitives (like **Lattice Crypto**) to resist quantum computers.

Man In The Middle (MITM)

The classical version of Diffie-Hellman is vulnerable to active attack, as it only guarantees key agreement, but not authenticity (verifying who you are agreeing with). If for instance Eve intercepts A from Alice and B from Bob. Eve can impersonates :

- Eve sends E_B to Alice. Alice compute K_{AE}
- Eve Sends E_A to Bob. Bob compute K_{BE}
- Eve shares a key K_{AE} with Alice and a separate key K_{BE} with Bob.
- She can read, modify and relay all communication.

Hence this implies a need for **trust** (authentication). DH solves the key distribution problem but create the identity verification one. The solution to this is that DH must be paired with a mechanism (like digital signature) to verify identity. The setup in this case requires a third party that needs to be trusted (a certificate authority) for public key ownership.

2.5.1 Authenticity

We have found a way to encrypt messages, meaning no one can see our messages from outside, but in the first place we have to make sure that our messages go to the right persons and vis versa, we need to be able to be sure that the message we have received is from the right person. Do to do, one need to define **authenticity**.

Definition 12. (Authentication) The process of verifying a claimed identity.

Ways to prove who you are

Traditionally, we have a lot of methods in order to authenticate ourself :

- **What you know**: password, secret key
- **What you are**: biometrics
- **What you have**: smart card, secure token

However now what can use many other ways to prove :

- **Where you are**: location, IP address
- **How you act** behavioural authentication
- **Who you know**: social ties
- ⋮

Public key cryptography

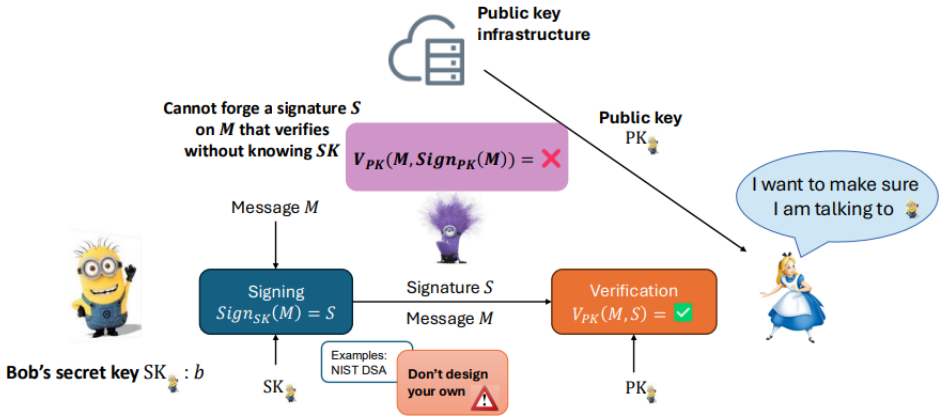
To go back on public key cryptography, the way it worked was that we needed a public key infrastructure which allowed us to expose our public keys to the world in order to communicate. To do so we need **digital signature**, **signing** and **verification**. The public key infrastructure needs to ensure authenticity.

Imagine now that Bob (secret key SK, public key PK) and Bob wants to communicate, he will send a message M and also a **signing** defined as

$$\text{Sign}_{SK}(M) = S$$

and send this back to Alice which she can verify using here own function V_{PK}

$$V_{PK}(M, S) = \checkmark$$



So how it works is the following

- Bob has a secret key. Using this key Bob performs a **digital signature** on the message.
- To verify a signature, Alice get Bob's public key from the repository and performs a verification function using the message, the signature, and Bob's public key. This function returns yes if the signature is valid, and no otherwise.

To be secure, and provide integrity, Digital Signature algorithms need to be such that it is not possible to create a Signature for a message that will verify under a given public key, unless you have the corresponding secret key. Under these properties, a digital signature enable Alice to know that she is talking to Bob: only Bob, that knows the secret key, could have created the signature.

Digital signatures

Digital signatures are computationally costly compared with most symmetric key algorithms of equivalent security, what this means concretely is that signing and encrypting is **slow** (not having to pre-share a key comes at a cost).

Naively, this gives us a big issue, having long message implies that the signing and verification will take a very long time and will be not practical, if you remember CS-200, you may remember the way to solve that issue :

2.5.2 Hash Functions

Hence, let us define three security properties that a hash function require

- **Pre-image resistance**: Given $H = h(M)$, it should be difficult to find M
- **Second Pre-image resistance**: Given M, difficult to find $M' \neq M$ such that $h(M') = h(M)$
- **Collision Resistance**: difficult to find any M, M' such that $h(M) = h(M')$
- **Deterministic**: Given M a hash function H will always output the same string $h = H(M)$

Cryptographic hash functions are designed to be fast and efficient to compute, and they have several important properties that make them valuable in various applications.



Figure 2.2: A beautiful diagram of a hash function (right?)

As always, one should never design those themselves and should use already existing hash functions

- MD5 (1991): 128 bits hash — insecure
- SHA0, SHA1: 160 bits — insecure
- SHA-2 (224/256 /384/512) — Ok but slow
- SHA-3 (224/256 /384/512)

The use cases of hash function are vast, in order to support digital signatures, build HMAC, password storage, file integrety, secure commitments, secure logging, blockchains, ...

Digital signatures on hash functions

So now instead of having a digital signature on the whole message (which can be enormous), one can sign using the **hash** of the message. Meaning that instead of having $\text{Sign}_{SK}(M) = S$, we will replace M by $H = h(M)$. Alice on the other side will receive both the signature and the message and also need to compute the hash of the message on her side in order to apply the verification.

So how it works is the following: only Bob has his secret key SK, so only Bob can produce a signature S that verifies under his public key PK. When Alice checks $V_{PJ}(h(M), S) = \checkmark$, she learns two things:

- **origin**: the signature was made by whoever holds Bob's SK, which should be Bob
- **Integrity**: M wasn't modified. Changing M changes $h(M)$, so the old signature no longer verified it.

All together

Now let us have both the encryption of the message and the signature so that we ensure, privacy and authentication. This implies that each users have **two pairs** of keys (2 × secret key, 2 × public key). In that case **confidentiality** is ensured as before with the ciphertext

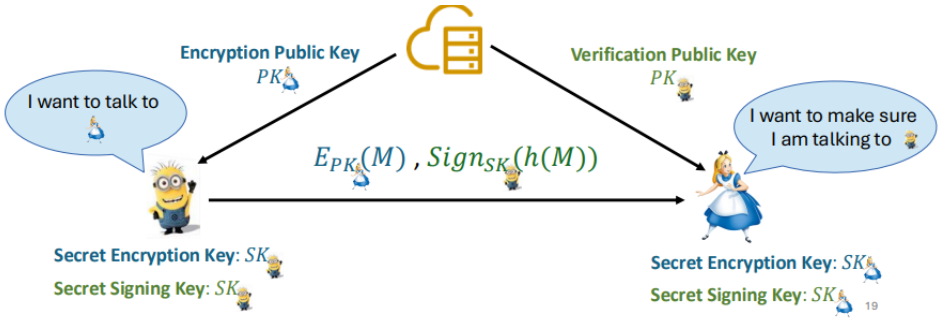
$$C = E_{PK}(M)$$

and the decryption computed as

$$D_{SK}(c)$$

The authentication on the other hand is using the other pair of keys with

$$V_{PK}(M, S) = \checkmark \text{ with } S = \text{Sign}_{SK}(M)$$



2.5.3 Integrity

Now we have again another problem **Integrity**

Definition 13. (Integrity) information cannot be **modified** by unauthorized parties

What this means is that if Bob sends '*Banana*' he can be sure that Alice will receive '*Banana*'.

Message integrity through digital signatures

As seen before in asymmetric cryptography, we have already seen a way to solve the integrity problem: digital signatures. Those provide

- Sender authenticity: since no party than the sender can create a valid signature for a message, the server can verify that the identity of the sender. No adversary can produce a valid signaature to the public key of the sender and therefore cannot impersonate the sender
- Integrity of the message: Since no party than the sender can create a valid signature for a message, no adversary can modify the message without being detected (if they modify the message, the signature would not be valid)
- Non-repudiation: Since no party than the sender can create a valid signature for a message, if a message has a valid signature, the sender cannot claim they did not sign it (no-one else can produce such signature).

However what this gives is that the signature S only verified if message M has not been modified

Integrity for Symmetric Cryptography

As in the asymmetric cryptography schemes, symmetric ones should also not have to suffer through integrity! Let us take the example of Block cipher, meaning the message is sent by block of fixed size, typically 128/256 bits. What this implies is that in order to send a message of size greater than a block, we require iteration, which implies that we need to create some sort of mode operation:

Mode 1: Electronic Code Book (ECB)

Straightforward scheme: encrypt and decrypt single blocks. So we work as a '*stream*' with each M_i being encrypted with $E_K(\cdot) \to C_i$. For which we can then reconstruct the message as

$$M = M_1 M_2 M_3 M_4 \cdots$$

Which each block with the same size as the key. However we have a big issue here imagine that $M_i = M_j$, what this gives is

$$\begin{aligned} M_i &\to E_K(\cdot) \to C_i \\ \underbrace{M_j}_{=M_i} &\to E_K(\cdot) \to C_i \end{aligned}$$

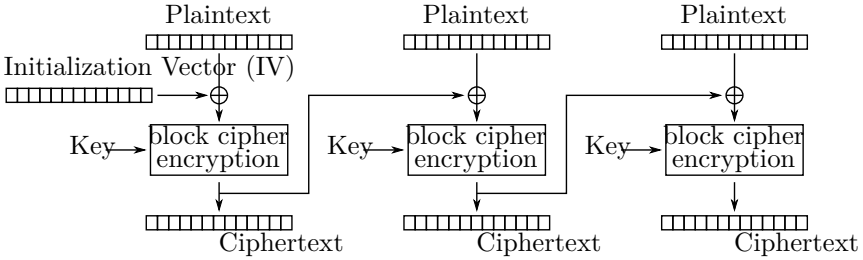
$$\text{if } M_i = M_j \text{ then } C_i = C_j$$

which implies that the attacker learns something about the plaintext without breaking the cipher. Confidentiality means the ciphertext should reveal nothing about the message except its length. With ECB, it reveals which blocks are equal and that alone is often enough to learn a lot.

Mode 2: Cipher Block Chaining (CBC)

The idea in this technique is to add some "*noise*" in the encryption by block in order to '*hide*' the equals blocks. We create an Initialization Vector (IV) which is a random block and the encrypt using the following rule

$$C_0 = IV$$
$$C = E_K(M_i \oplus C_{i-1})$$



Cipher Block Chaining (CBC) mode encryption

Where each block cipher encryption is $E_K(\cdot)$.

Remark Note that every time a block is encrypted, by the properties of the block cipher, the ciphertext is random. Because the IV and/or the plaintext changes every time, this random value is different every time. Thus, when XORed with the next plaintext it creates a random string. If the IV changes, this is similar to a one-block one-time pad, but if one reuses the IV for the same plaintext and key, this value will be repeated.

Decryption of CBC

In order to decrypt messages all we need to do is work the other way around. Recall that $C_0 = IV$ and that $C_1 = E_K(M_1 \oplus IV)$. Decrypting it gives $D_K(C_1) = M_1 \oplus IV$ which means that we can recover M_1 as $M_1 = M_1 \oplus IV \oplus IV$. Hence, given $C_i = E_K(M_i \oplus C_{i-1})$, Alice can decrypt it as $D_K(C_i) = M_i \oplus C_{i-1}$, by applying $\oplus C_{i-1}$ to it.

$$\begin{aligned} M_i &= D_K(C_i) \oplus C_{i-1} \\ &= M_i \oplus C_{i-1} \oplus C_{i-1} \\ &= M_i \end{aligned}$$

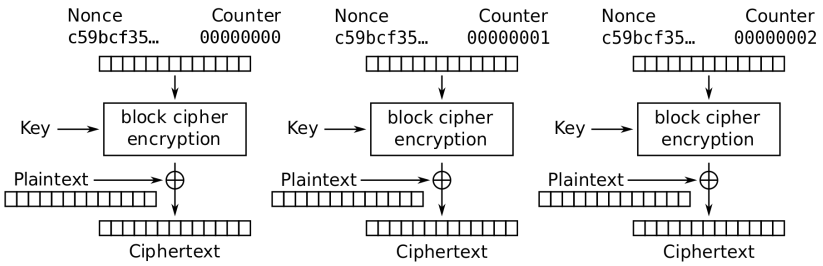
However this raises some questions: what if IV is incorrect? Is the full decryption wrong? Can you decrypt a block alone? What do you need?

If the IV is incorrect, only the first block is corrupted, Recall that the rest only depend on the ciphertext. Note also that blocks cannot be decrypted without having the previous ciphertext!

Mode 3: Counter Mode (CTR)

The idea here it to use increasing nonce to add randomness without dependencies between block. What this means is that the CTR encryption is defined as

$$C_i = E_K(\text{Nonce} + i) \oplus M_i$$



Counter (CTR) mode encryption

Now the question is how do we decrypt this cipher? Well if you think about it:

$$M_i = E_K(\text{Nonce} + i) \oplus C_i$$

right? Let us see if this is true

$$\begin{aligned} E_K(\text{Nonce} + i) \oplus C_i &= E_K(\text{Nonce} + i) \oplus E_K(\text{Nonce} + i) \oplus M_i \\ &= M_i \end{aligned}$$

Block Ciphers

Strengths

- **High diffusion**: information from one plaintext symbol is diffused into several ciphertext symbols
- **Immunity to tampering**: difficult to insert symbols without detection

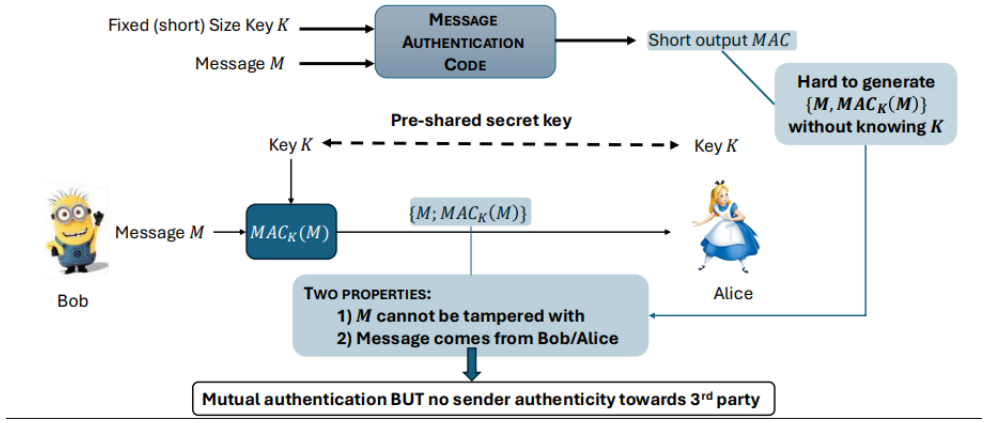
Weaknesses

- **Slow**: an entire block must be accumulated before encryption/ decryption can begin
- **Error propagation**: in some modes of operation errors affect several bits/blocks

Summary

- **Electronic Code Book (ECB)**: Directly encrypt and decrypt single blocks. Large information leakage due to lack of randomness across block's encryption
- **Cipher Block Chaining (CBC)**: Avoids ECB problems by using previous blocks to add randomness to every encryption. Propagates errors and prevents single-block decryption
- **Counter mode (CTR)**: Uses an increasing '*nonce*' to introduce randomness across block's encryptions. Enable parallel encryption and decryption.

What about integrity? As in the asymmetric case, let us see how to ensure integrity for symmetric encryption. To do so we'll use **message authentication code** (**MAC**).



A MAC is like a hash function with a **key**. It takes a shared secret K and a message M and outputs a short tag $MAC_K(M)$. Alice computes $T = MAC_K(M)$ and sends $\{M, T\}$.

Bob then receives $\{M, T\}$ and recomputes $MAC_K(M)$ himself using K, and checks whether it equals T.

But first let us be a bit more '*formal*' about the defintion of our MAC. A message authentication code receives two inputs:

- A small key that must be kept secret
- The message who integrity needs to be secured

The MAC algorithm outputs a short string that helps Alice to verify the integrity of the message sent by Bob.

The key property of the MAC is that it is very difficult to produce a pair [message; $MAC(k, \text{message})$] without knowing the key.

This is the symmetric counterpart of a signature check. The difference is that Bob can't merely verify: he recomputes the exact same value, because he hols the same key. The integrity here comes from the fact that if Eve changes M, the tag no longer matches and she can't compute a new one without K. The authenticity on the other hand comes from that only someone who knows K could have made the tag, so it came from Alice or Bob.

Repudiation

Now imagine an exchange between Alice and Bob with Charlie being a third party observes. Even though Alice and Bob know they are talking to each other (only any party with knowledge of K could have produce a valid MAC, message pair, they both could have produced the message.)
So the message can be repudiated which means both A and B can claim it was the other who wrote the message.

Example MAC: CBC-MAC

Let us see how to turn a block cipher into a mac. To do so CBC MAC defines the following encryption methods

$$C_0 = 0[\text{ any fixed } IV]$$
$$C_i = E_K(M_i \oplus C_{i-1})$$
$$MAC_K(M_1 \dots M_x) = C_n$$

The difference here with CBC encryption is that CBC-MAC is deterministic, also only the output of CBC-MAC is the last block.

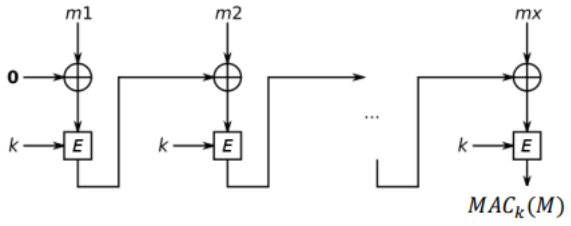
Remark

Remark that CBC can be turned into a MAC by:

Fix IV (e.g. 0) and then do CBC. The MAC is the output of the last encryption block: one could have only gotten there for one message and one key. Note that the last block cannot be used to recover anything about the message: it looks random (as it is the output of a block cipher)

Limitation

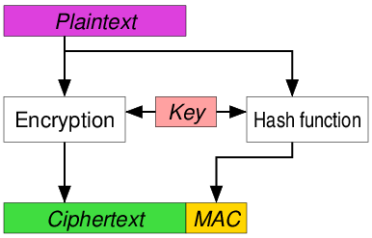
The encryption scheme is only secure if the length of the message M is known.



And all together now ...

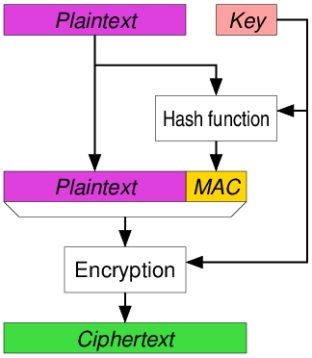
Encrypt and MAC

- × No integrity on the ciphertext → Cipher can be attacked which therefore mean that we need to decrypt to know if the ciphertext is valid
- ✓integrity of the plaintext can be verified
- × May reveal information about the plaintext → repeated messages, recall the IV of the MAX is fixed (can be solved with a counter)



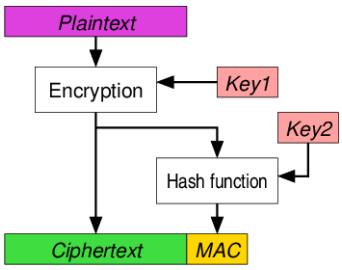
MAC then encrypt

- × No integrity of ciphertext (in theory) it is possible to change ciphertext and have a valid MAC
- ✓Integrity of the plaintext can be verified
- ✓No information on the plaintext either, since it is encrypted



Encrypt then MAC

- ✓Integrity of ciphertext → ensures you only read valid massages! Cipher cannot be attacked!
- ✓Integrity of the plaintext can be verified
- ✓No information on the plaintext either, since it is encrypted



2.6 Authentication

What is authentication?

Definition 14. (Authentication) The process of verifying a claimed identity

Remark 7. Be careful: authentication must not to be confused with **message authentication** in which we verifying

- Message provenance
- Message integrity

Where does Authentication fit?

The question that we need to answer first before thinking of the how is the where, where should authentication fit? From what we have seen in the past week (weeks 2 and 3), we know that between the path from a request to the completion by the TCB have authorization, access control. Recall that **access control** establishes whether a principal is authorized to operate on an asset. Before the access control mechanism can make this decision, **authentication** needs to happen to ensure that the principal is who he says he is.

Remark We will mostly cover authentication of **users**, and a bit of machine authentication.

Where to prove who you are

There is a lot of way to prove who we are, there are traditionals one with

- **What you know** password, secret key
- **What you are** biometrics
- **What you have** Smart card, secure token

And some more modern

- **Where you are** location, IP address
- **How you act** behavioural authentication
- **Who you know** social ties

And many others ...

What you know: Passwords

Definition 15. (Password) Secret shared between user and system

It works by the user having a secret password and the system checking it to authenticate the user. However this gives us a lot of problems to solve

- **Secure transfer**: the password may be eavesdropped when communicated
- **Secure check**: naive checks may leak information about the password
- **Secure storage**: if stolen the full system is compromised
- **Secure passwords**: easy-to-remember passwords tends to be easy to guess

The secrecy of the password depends on how easy it is to predict. If the password can be guessed, it does not matter all the above is secured. The password is still not secret.

Secure transfer

Well as seen before, if we were to sent the password in clear through the communication channel, anyone with an access to it could impersonate the user. Hence as we have already seen in the class, what we require here is to have a secure transfer: encrypt the tuple $(\text{login}, \text{password})$ in transit so that it cannot be eavesdropped. Typically one does not just encrypt this information, one also **encrypt the channel** using a standard protocol (e.g. TLR, HTTPS which combines the concepts we have seen in a safe way). As always do **NOT** build your own protocol.

Beware of replay attacks

Well, encrypting the password may not be enough here right? Imagine if an eavesdropper just copied the encrypted message that Alice has sent to Bob and resend, Bob wouldn't have any way to see whether or not it was Alice or Eve. Even if the password is encrypted, one can copy the content and then reply the encrypted content.

Challenge-Response protocols

To avoid reply attacks (for passwords and in general) one usually uses **Challenge-Response** protocols. In these protocols:

1. The principal that wants to authenticate first declares their intention to login
2. The party authenticating the principal sends a **challenge**, a random number (from a large space, to ensure that there are no repetitions and cannot be predicted)
3. The principal encrypts the password together with the challenge. This ensures that
   (a) The password is never encrypted to the same value
   (b) the encryption is fresh (i.e., corresponds to the declaration in step 1)

After checking the password, the authenticating party **must** delete the challenge so that the message cannot be replied.

When Bob receives the tuple, in order to be sure that this tuple indeed authenticates Alice, Bob needs to know what is the correspondence between Alice and her password. This is typically stored in some sort of database. And as always, if the password are stored in the clear, an adversary can read them (e.g., in UNIX the password file is readable by anyone!) and then use them to impersonate the users. Also, remember the asymmetry of the adversary vs. the defender. We do not need all adversaries to be able to read the passwords or breach into the database. It suffices that one attacker gets access and leaks the passwords of millions of people (fortunately most of these database are not in the clear, although that does not mean they are safe). A first idea can be to encrypt the passwords under storage. When Bob receives the password, decrypts it, and check the challenge. If the challenge is fresh, then to check if it corresponds to the encryption stored in the database he can

1. Encrypt the received password and compare it to the stored value
2. Decrypt the stored password and compare it to the encrypted value

But note that the encryption is as secure as the security of the key. To encrypt/decrypt the passwords the key needs to be on the same machine, so if the adversary can access the database, he can also steal the key. Hence we need a primitive that does not require a key and allows for checking, a **hash function**. When Bob receives the password, he decrypts it, and checks the challenge. If the challenge is fresh, Bob hashes the received password and compares it to the stored value.

In that case, what is important is that if the adversary gets access to the database, he cannot recover the password from the hashes: i.e., the hash function must be pre-image resistant. Okay this is nice BUT, does password are random, not really, we use mostly letter (upper and lower) and punctuation symbols. The space is not inexplorable. And also, among those, not every password is equally likely. Hence one can compute the hash of the most typical passwords and start trying them. More tricks are available in order to speed up that process (e.g. parallel computing, rainbow tables, etc.)

Hash and salt

The better and the one we actually use is to store the password as a *hash + salt*. When using salts, one:

- chooses a salt for every password
- hashes the password concatenated with the corresponding hash → Note that this means that a password will look different when salted with different values
- stores the hash together with the salt (the salt is in the clear)

When verifying the password Bob receives the password, decrypts it, and checks the challenge. If the challengs is fresh, Bob finds the corresponding salt (the one in Alice's entry) in the database, concatenates it with the received password and computes the hash. Then compares the result to the stored value.

Remark Note that breaking one password is still possible. As the salt is in the clear one can try to compute the hash of all possible passwords with that hash. So one targeted attack is equally cheap than before, but retrieving the passwords for all entries in the database is very expensive.

In reality, people do even more things:

- Instead of using one normal hash function designed to be fast, use one that is slow, so as to slow down the adversary that tries to compute the dictionary. Then repeat several times, not because it is more secure, but because it slows the attack even more. **Important**: slow down means that if one needs to compute this for many passwords the difference is noticeable, but for every login check there is virtually no different.

- Ensure that people do not choose typical passwords and require specific elements so that they are random and it is harder to guess them.
- Require a server to do a computation. This means that the adversary cannot compute an offline dictionary
- And of course, hinder as much as possible access to the passwords database! Access control is your friend.

2.6.1 Secure checking

Now the question which seems at first kind of silly, '*how do we check?*', meaning that given a password how should we check if it is correct? Is doing the following secure :

```
String real_password = "hello world";
def check_pwd(password):
boolean check_pwd(String password) {
        for (int i = 0; i < password.length(); i++) {
                if (real_password.charAt(i) != password.charAt(i)) {
                        return false;
                }
        }
        return true;
}
```

However here, the check ends reveals how many letters are correct. So you should always try to take the same time regardless of when it fails.

```
String real_password = "hello world";
def check_pwd(password):
boolean check_pwd(String password) {
        boolean flag = true;
        for (int i = 0; i < password.length(); i++) {
                if (real_password.charAt(i) != password.charAt(i)) {
                        flag = false;
                }
        }
        return flag;
}
```

Here regardless on the input the time of the check stays the same.

Remark 8. Note that with hashing this happens by default!

Authentication Library

As always do not program your own password checker, there are many authentication libraries out there!

- Dedicated security frameworks (apache SHIRO)
- Embedded authentication libraries in web frameworks
- Cross-platform authentication libraries
- OAuth: performs the authentication in a third party

Remark Oauth is very convenient, but implies to leak information to third parties about which users log into your service and whe.

Problems with passwords
All the work we have done above is about storing password, however strong passwords are difficult to remember, maybe we have to have them written, reuse across systems... Hence they can be stolen

- Keylogger
- Shoulder surfing
- Phishing
- Social engineering

Hence we might want to look into better option on how to prove who we are, recall the start of the lecture with the traditional way to prove who we are

- **What you know**: a secret that only you can know, and therefore proving knowledge of the secret ensures that it is you (e.g. password)
- **What you are**: a trait that is inherent to you and non-one else had, and therefore proving this trait ensures that it is you
- **What you have**: object that only you can have, and therefore proving possession ensures that it is you

In modern life, the traditional means are augmented by other traits are often unique (some time called soft biometrics)

- **Where you are**: applications detect your common locations and trigger alarms if you are far
- **How you act**: how you type, how you move the mouse, how you pressure the keyboard
- **Who are your friends**: social networks tend to be unique

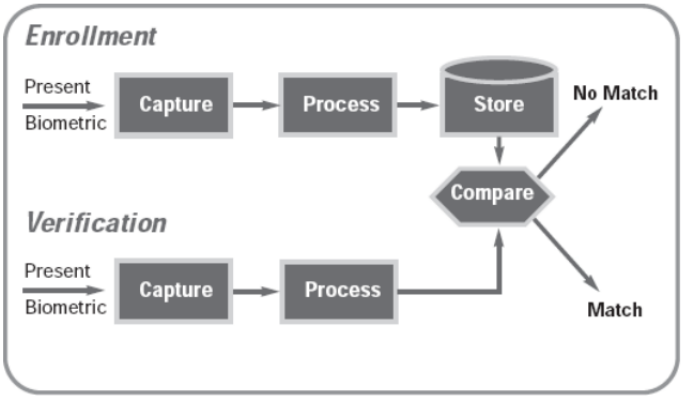
Definition 16. (Biometrics) is the measurement and statistical analysis of people's unique physical characteristics

To avoid having to remember, another authentication mean is Biometrics. These are unique physical characteristics that can be extracted and measured such that they can be used as a "secret" for authentication.

Many of them do not require users to act, they can be checked passively. They are also very difficult to delegate, as they are physical.

Enrollment

The biometric authentication has two phases. The first one is **enrollment**, which is when the biometric of the user is introduced in the system and associated to their login. The biometric is captured by a sensor (fingerprint reader, camera, etc.), and processed to convert it into a stream of bits. This stream is called biometric template.



Verification

The second phase is Verification. Here the biometric is captured and processed as before, and the template obtained is compared to the stored one. Depending on which the following three phases happen we different security and privacy trade-offs: Capture: the sensor captures the biometric, Process: the captured biometric is converted on the template, Store: the template is stored

- Local: in the device where the assets that the biometrics protect are stored
- Remote: on a server remote from the assets being protected

Storing/processing locally is more privacy-preserving, but harder to secure and difficult to update. Storing/processing remotely is less privacy-friendly (the server sees the biometrics), but is easy to secure and update.

What you are: Biometrics

As opposed to the hash in passwords, this comparison is not exact. Because the capture may contain differences (e.g. dirt of fingers, light when taking photos, etc), one has to decide a threshold to define what a match means.

When deciding this threshold one has to make a balance between false positives (how many times a biometrics that is not from the user is considered a match) and false negatives (how many times the user provides her biometrics, but these are rejected). What balance depends on the assets you are securing and what is more important, to keep them safe or to keep users happy.

2.6.2 Tokens

Problems with Biometrics

Hard to keep secret	Signature on ID card, fingerprint left on glasses, door handles, ..., photos! All of those require liveness detection: Liveness detection are measured in the capture/processing steps in which the users are required to do actions to demonstrate they are actually alive.
Revocation is difficult (impossible?)	Sorry, your irise has been compromised, please create a new one...
Identifiable and unique	Linking across systems
May reveal private information	• Iris → disease • Face → Identity
Not always universal or immutable	Fingerprint disappear, iris changes with lenses, ...

After having see all of those issue let us try a new approach **tokens**. Recall from before that we also have authentication by proving ownership of a token. This proof can come by having a chip running a protocol on a secret. For instance, a smart card signs a challenge sent by the ATM to prove knowledge of a key.

Or from a device that contains a secret shared with the server. These device output a number based on this secret and time (require synchronization). The device is authenticated when it outputs the same number as the server.

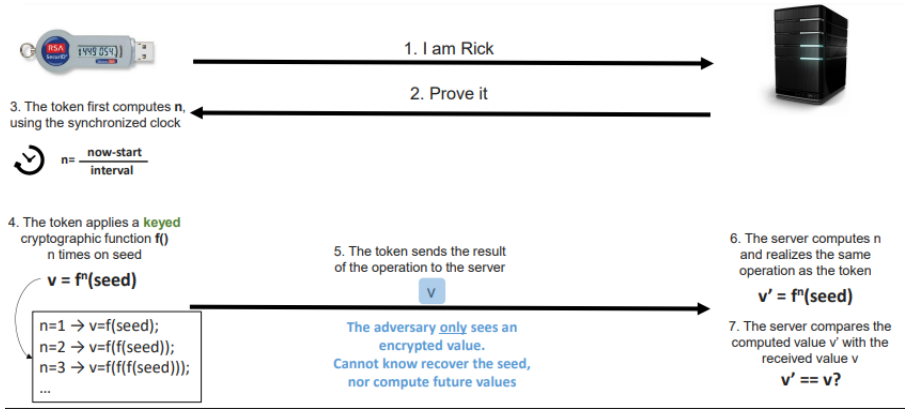
Authentication

How it works could be the following

1. Step 1, Offline initialization: token and server establish a common *seed* and synchronize their clocks
2. From then on, Operation: obtain a random number from the seed that can only be computed by the token

The server asks to prove, the token computes which interval it is with respect to the time it was synchronized with the server.
Then applies a keyed cryptographic function using as key the secret shared with the server n times on the seed, and sends the result to the server. Notice that from this value an adversary cannot recover the seed or the key (properties of encryption),

and cannot compute future values because she does not have the key. The server does the same operation and checks



Remark 9. The reason we use a keyed function and not a hash is that everyone can compute a hash! If we use a hash instead of a keyed function, given v I can produce any future v by hashing the value!

Definition 17. (2FA - Two factor authentication) Combine two out of the three factors: (What you know, what you have, what you are)

What machines have: Secret key Finally, when we authenticate machine and not users, this is done using public key cryptography (e.g., to authenticate web servers). The authenticating party signs with their secret key. We use the secret keys to produce **digital signatures** to authenticate parties (e.g., used in the internet protocol HTTPS/TLS to authenticate the **server** (and can be used also to authenticate the client))

Remark 10. Building authentication protocols is **hard**

- Defending from **man in the middle** — Use signatures
- Defending from **replay attackscA — uses challenges / nonces**

Even with that, it is still difficult to get right! One should use well established protocols (TLS 1.3, ISO 9798-3)

2.7 Adversarial thinking, Attacks and defenses

Why do we study (and simulate) attacks

The first reason is a **deeper understanding of defense**: Very good attackers make very good defenders (and vice versa). Mediocre attackers, make extremely poor defenders (find some attacks...).

Another reason is job opportunity: penetration testing (pentesting) is a **major** industry. However always recall that the lack of found attacks does **not** guarantee that the system is secure!

Remark Remember you cannot freely hack around...

The security engineering process

Recall the following process

1. Define a security policy (principals, assets, properties) and a threat model. However here the adversary can exploit
   - Misidentified principals, assets, or properties
   - Capabilities beyond what is considered in threat model (more access or more computational/algorithmic capabilities)

2. Define a security mechanisms that support the policy given the threat model. Adversary can exploit
   - Design weaknesses/flaws in the security mechanisms
3. Build an implementation that supports/ embodies the mechanisms. Adversary can exploit
   - Implementation or operation problems that allow you to subvert mechanisms

The attack engineering process

The goal here it to exploit misidentified assets in the security policy.

Example 1 - Extracting keys from Hardware Secure Modules (HSMs)

Given the following policy: HSMs implement PKCS#11 standard for interoperability, there is an API to create **internally** a new key from the secret key: Given `bits_length` and `offset`, it uses `bits_length` of the secret key from position `offset`.

Here let us assume a strong key exists in the HSM. We then ask the HSM to derive a new key of length 1 byte at offset 0. We then use the new key to do an operation, say HMAC on a known input (allowed by the HSM), we can then brutefors the key (input known, output known, key only 1 bytes) and then repeat with keys at different offsets until we have discovered the full key.

Example 2- From cable to the Air

Here we have an Engine Control Units (ECU) that controls the vehicle. ECU is connected to GSM/WiFi which gives a remote adversary access to the CAN bus and all the (safety) functions of the vehicle

Example 3 - IoT devices are a Weak Link

IoT weakly protected devices connected to internet can lead to hackers compromising the Smart Grid with 100k devices.

Example 3 Unilateral User Authentication in GSM

When GSM was designed atenna (Base Trasceiver Stations - BTS) they were difficult to implement and expensive to build. Thus, operators decided that the network did not need to authenticate! Nowadays commodity hardware can be used to fake base station and perform a min in the middle.

Example 4 - the ML and AI revolution

The power of inference at your fingertips! Apparently irrelevant information becomes critical for the security system. Break better and faster!

Machine learning substitutes complex modelling tasks by data collection, and modern LLM-based AI enables easy vulnerability discovery and exploitation. However that revoluion **also** works for the good guys!!

- Improved malware detection
- Predicting zero days (unknown vulnerabilities)
- Identifying vulnerable devices
- Automated log analysis

Exploiting security mechanisms design weaknesses

Example 1 - Weak cryptographic primitives

Tesla Model S – Key Fob algorithm to start the car allows to recover key in seconds (with pre-computation) GSM – A5/1 and A5/2 weak allow ciphertext only attacks Can be real time by FPGA parallel computation!

In both cases the algorithms were secret, but researchers reverse engineered them. Once the algorithms were known researchers identified vulnerabilities that allowed them to decrypt and read messages, and even recover the key

Security by obscurity is a bad idea — Open design principle

Exploiting bad operation decisions to subvert security mechanisms

Example 1 WEP bad use of RC4

WEP uses RC4, a secure stream cipher when the IV is random.

In WEP the IV is defined to have 24 bit. The implementation uses this 24 bits in such a way that the IV is repeated every 5000 / 6000 frames! Adversary can accelerate the attack by spoofing MAC addresses to ask for more frames

Exploint implementation flaws to subvert security mechanisms

Example 2 - Bugs, bugs and more bugs

Programmers make mistakes

- They forget checks, or check the wrong thing
- They do not sanitize, or do not sanitize correctly
- They forget to protect what needs to be protected
- They get confused about origin or reliability of data/ variables (Ambient authority and confused deputy)

2.7.1 Reasoning as a defender

Idea: help security engineers reason about threats to a system: "*What can go wrong?*"

Threat modelling

Definition 18. (Threat modelling) Process to identify potential threats and unprotected resources with the goal of prioritizing problems to implement security mechanisms.

Systematic analysis

- What are the most relevant threats?
- What kind of problems can threats result in?
- Where should I put more effort to protect?

Attack trees

The attack goal is the root, and the ways to achieve this goal are represented by the branches. The leafs are the weak resources

STRIDE

Identify system entities, events, and the boundaries of the system. Reason about threats enumerating the type of threats that can be embodied by the adversary

P.A.S.T.A.

Start from business goals, processes, and use cases.
Find threats within business model, assess impact, and prioritize based on risk

Reasoning about attacks STRIDE (by Microsoft)

Threat	Property threatened	Example
Spoofing	Authenticity	A member of the council of Rick convinces Morty that he is
Tampering	Integrity	The bad minion modifies the plan message send by Gru to our fav
Repudiation	Non-repudiation	Summer denies have told Morty that Rick was waiting
Information disclosure	Confidentiality	Summer learns about the secret plan of Rick and M
Denial of Service	Availability	The minions flood Dr. Nefario's lab with bananas and he cannot recei
Elevation of Priviledge	Authorization	Bob the minion gain access to the system with Gru's cre